



Laboratory 3 – Interrupts, low power and watchdog timer

1 Overview and Goals

In the previous laboratories we have explored digital input and output on the MSP430F5438, where we took input from the push button switches and utilized this to control the LEDs on the board. To check whether we had received input we constantly checked within a tight loop to see if the button had been pressed. This method is referred to as ‘polling’, as we are constantly checking within software for a change in the input. As we have seen this method can be used effectively, but it requires the processor to be constantly working. Within this laboratory we will explore a method that allows the processor to only respond to a change on input.

There are several specific objectives in this lab:

1. Investigate in further detail the operation of the GPIO ports on the MSP430F5438
2. Understand the function and use of interrupt based software.
3. Discover the low power options available with the MSP430F5438.
4. Use further control logic to manipulate output with interrupt based software.

2 Introduction

Polling is to constantly check or “poll” something. This can be applied in software for example to any user interface where the microcontroller can poll the buttons to verify if there are any user inputs, as we have seen previously. On the other hand, **Interrupts** use hardware to implement a more efficient way of letting the firmware know of any occurring events.

Interrupts are like a telephone. Consider these two situations. You are at your house and you are expecting a call. You could periodically grab the telephone every 5 seconds and check if there is anyone in the line calling you. Yet, that would mean that you would waste a lot of time *polling* the telephone line without any success. On the other hand, you can continue whatever you are doing and when the telephone rings you receive an *interrupt*. Once it rings, you leave whatever you’re doing and take care of the telephone (*you service the telephone interrupt*).

This is completely analogous to a microcontroller. Once a microcontroller receives an interrupt request it suspends its current execution and jumps to the interrupt service routine, and once finished, continues at its last executed instruction (in the main routine).

Now extending on this idea, imagine you are at home and your house phone rings at the same time as your mobile phone. You now required to make a choice as to which to answer, the same could occur with interrupts within a MCU, therefore in the same way that you would use a priority based scheme to decide which call to answer, interrupts service routines are given priorities also.

The MSP430F5438 provides various interrupt sources which are detailed in page 13 of the datasheet.

Lab Question 1. From the functional block diagram within the datasheet for the MSP430F5438, which two Input/Output ports have Interrupt Capability?

Lab Question 2. From Table 2 within the datasheet, which interrupt within the MSP430F5438 has the highest priority?

Lab Question 3. By performing an appropriate internet search, detail what is the difference between Maskable and Non Maskable interrupts?

2.1 Useful Documentation

To help new developers get up to speed with the rich set of functionality that is available with the MSP430 there exists extensive documentation, relating to both the MCU and the Experimenter board. The following wiki is populated by TI and is a good place to begin to find documentation and software examples.

http://processors.wiki.ti.com/index.php/MSP-EXP430F5438_Experimenter_Board

The following documents (pdfs) will be required for this and any subsequent laboratories, they are available at the following website and copies are available on GCU Learn.

<http://www.ti.com/tool/msp-exp430f5438>

- MSPF5438 Experimenter User Guide (Rev G.)

<http://www.ti.com/product/msp430f5438a>

- MSP430F543xA, MSP430F541xA Mixed Signal Microcontroller (Rev. B)

Datasheet for the MSP430F5438

<http://www.ti.com/lit/ds/symlink/msp430f5438.pdf>

3 Digital Input/Output Initialization

In the previous laboratory, we have setup Port 2.6 as an input port by setting the data direction register to configure the port as an output, and setting up the selection register to select digital I/O rather than utilizing a special function register.

```
// Setup P2.6 as an input port
P2DIR = P2DIR & ~BIT6;
P2SEL = P2SEL & ~BIT6;
```

We then setup the pullup resistors, by enabling pull ups then setting the pull up to the up configuration.

```
// Enable the pull up resistor and set as a pullup
P2REN = P2REN | BIT6;
P2OUT = P2OUT | BIT6;
```

As this is something we may wish to do commonly, it makes sense to place this software into a peripheral initialization function, that we can call at the start of our programs. This promotes software reuse, and will save time and effort later. The following is an example function that will setup S1 to operate as you used in the previous laboratory.

```
void periphInit(void)
{
    // Setup P2.6 as an input port
    P2DIR = P2DIR & ~BIT6;
    P2SEL = P2SEL & ~BIT6;

    // Enable the pull up resistor and set as a pullup
    P2REN = P2REN | BIT6;
    P2OUT = P2OUT | BIT6;
}
```

Lab Question 4. Update the function above to setup s2, and LEDs 1 and 2 appropriately.

3.1 Interrupts

Interrupts are asynchronous breaks in normal program flow that occur as a result of events outside the running program. They are usually hardware related, stemming from events such as a button press, the completion of a data transfer or the expiration of a timer. We can see from these examples that interrupt conditions are independent of particular instructions; they can happen at any time, stopping normal program execution only when necessary. Interrupts trigger execution of instructions that perform work on behalf of the system, but not necessarily the current program.

Within an MCU interrupts are commonly used for a range of applications:

- Urgent tasks that must be executed promptly at higher priority than the main code. However, it is even faster to execute a task directly by hardware if this is possible.
- Infrequent tasks, such as handling slow input from humans. This saves the overhead of regular polling.
- Waking the CPU from sleep. This is particularly important in the MSP430, which typically spends much of its time in a low-power mode and can be awakened only by an interrupt.

In the metaphorical example above, you required to pick up the phone only once you had been alerted to it. Once we have been alerted to the presence of an interrupt the MCU will require to take some action. The action to be performed in response to an interrupt is written by a programmer just as normal software is created. It is performed within what is referred to as an *interrupt handler* or an *interrupt service routine* (ISR). An ISR is like a function but with the critical distinction that they are requested by hardware at unpredictable times rather than called by software in an orderly manner determined by the programmer.

When an interrupt occurs the processor will perform the following sequence.

1. Interrupt signal is sent by the device to the processor .
2. If the interrupt line is enabled the following sequence of events occur in the system, else the interrupt is ignored. The processor completes its present instruction (if any) .
3. It stores the address of the next location and content of status register to the stack
4. It informs the device that its request has been granted and in response the device de-activates its IRQ.
5. Using some suitable technique the processor loads its program counter(PC) with address of the ISR.
6. With return statement occurring at the end of the ISR all stored content is loaded back into the respective registers and the processor resumes its suspended program

3.2 Interruptable Ports

As we have seen in the previous laboratories to setup the operation of I/O ports, we set values within registers. So far the registers we have used are

- PxDIR
- PxOUT
- PxSEL
- PxREN
- PxIN

Lab Question 5. Write a sentence detailing the function of each of the above registers. (PxOUT has two functions)

Each pin of ports P1 and P2 is able to generate an interrupt request (pin is interruptible) and is configured using the PxIFG, PxIE, and PxIES registers. The port makes use of all the same configuration registers as non-interruptible ports (as described above), but with three additional registers:

Interrupt Enable (PxIE)

- ☐ Read-write register to enable interrupts on individual pins;
- ☐ PxIE configuration:

- Bit = 1: The interrupt is enabled;

- Bit = 0: The interrupt is disabled.

- ❑ Each PxIE bit enables the interrupt request associated with the corresponding PxIFG interrupt flag;
- ❑ Writing to PxOUT and/or PxDIR can result in setting PxIFG.

Interrupt Edge Select Registers (PxIES)

- ❑ This read-write register selects the transition on which an interrupt occurs for the corresponding I/O pin (if PxIE and GIE are set);
- ❑ PxIES configuration:
 - Bit = 1: Interrupt flag is set on a high-to-low transition;
 - Bit = 0: Interrupt flag is set on a low-to-high transition.

Interrupt Flag Registers (PxIFG)

- ❑ The bit of this read-write register is set automatically when the programmed signal transition (edge) occurs on the corresponding I/O pin, provided that the corresponding PxIE bit and the GIE bit are set;
- ❑ Each PxIFG flag can be set by software, enabling an interrupt generated by software;
- ❑ Each PxIFG flag must be reset with software;
- ❑ PxIFG configuration:
 - Bit = 0: No interrupt is pending;
 - Bit = 1: An interrupt is pending.

Following the same procedure that was utilized in the first two laboratories.

1. Create a new project, named pushInterrupt.
2. Add a main.c file to the project.
3. Enter the following code into the main function.
4. Compile the code.
5. Download the code onto the board.
6. Run the code.

```

//*****
// Description:  Software to demonstrate interrupt based digital input on
//               push buttons on P2.6
// Author:
// Date:
//*****
#include "msp430x54x.h"
void main(void)
{
    long int i=0;

    // Stop WDT
    WDTCTL = WDTPW + WDTHOLD;

    // Setup P1.0 as an output port
    P1DIR = P1DIR | BIT0;
    P1SEL = P1SEL & ~BIT0;

    // Setup P1.1 as an output port
    P1DIR = P1DIR | BIT1;
    P1SEL = P1SEL & ~BIT1;

    // Setup P2.6 as an input port
    P2DIR = P2DIR & ~BIT6;
    P2SEL = P2SEL & ~BIT6;

    // Enable the pull up resistor and set as a pullup
    P2REN = P2REN | BIT6;
    P2OUT = P2OUT | BIT6;

    // P2.6 Hi/lo edge
    P2IES |= BIT6;

    // P2.6 IFG cleared
    P2IFG &= ~BIT6;

    // P2.6 Interrupt Enable
    P2IE |= BIT6;

    // Enable interrupts
    _EINT();

    // Polling loop with software delay
    for(;;)
    {
        // Toggle LED 2
        P1OUT ^= 0x02;
        // Spin round for n cycles
        for(i=0;i<30000;i++){;}
    }
}

```

```
// Port 2 interrupt service routine
#pragma vector=PORT2_VECTOR
__interrupt void Port_2(void)
{
    // P1.0 = toggle LED 1
    P1OUT ^= BIT0;

    // P2.6 IFG cleared
    P2IFG &= ~BIT6;
}
```

In the above software the following lines setup the interrupt for the push button s1.

```
// P2.6 Hi/lo edge
P2IES |= BIT6;

// P2.6 IFG cleared
P2IFG &= ~BIT6;

// P2.6 Interrupt Enable
P2IE |= BIT6;

// Enable interrupts
_EINT();
```

Lets look at each of these individually.

The first line is setting up the transition type that will cause an interrupt, in this case a high to low transition.

```
// P2.6 Hi/lo edge
P2IES |= BIT6;
```

Lab Question 6. What other registers have we set in this lab and in the previous lab, to ensure that pressing the switch will cause a high to low transition?

The second line clears the interrupt flag within the register for that pin. When an interrupt occurs, a flag (bit within a register) is set high. The programmer can utilize this flag to see which bit was set within an interrupt handler and take action accordingly.

```
// P2.6 IFG cleared
P2IFG &= ~BIT6;
```

The third line enables the interrupt for the specific bit on the port.

```
// P2.6 Interrupt Enable
P2IE |= BIT6;
```

The fourth line is the global interrupt enable. The interrupts are activated using `eint()`; At this point you might be asking yourself: Wait, I activated the interrupt when I called the line with `P2IE`. Why do I have to enable it again? The answer lies in the name of these interrupts. These interrupts are maskable, which means that although they are controlled individually, a second layer exists which can enable or disable them together. Therefore, we must disable the masking to allow our interrupts to run. The major benefit of the interrupt solution is that code continues to be executed after the interrupt is enabled. Rather, the for loop is executed. Whenever the user presses on the button, the interrupt handler is executed and then the CPU returns to its state of execution before the interrupt occurred.

Lab Question 7. Change the above code so that LED 1 toggles when switch S2 is pressed using an interrupt

3.3 Operating Modes

The MSP430 has one active mode and five software selectable low-power modes of operation. An interrupt event can wake up the device from any of the low-power modes, service the request, and restore back to the low-power mode on return from the interrupt program. These low power modes allow the system to maximize battery life.

The following six operating modes can be configured by software:

- Active mode (AM)
 - All clocks are active
- Low-power mode 0 (LPM0)
 - CPU is disabled
 - ACLK and SMCLK remain active, MCLK is disabled
 - FLL loop control remains active
- Low-power mode 1 (LPM1)
 - CPU is disabled
 - FLL loop control is disabled
 - ACLK and SMCLK remain active, MCLK is disabled
- Low-power mode 2 (LPM2)
 - CPU is disabled
 - MCLK and FLL loop control and DCOCLK are disabled
 - DCO's dc-generator remains enabled
 - ACLK remains active
- Low-power mode 3 (LPM3)
 - CPU is disabled
 - MCLK, FLL loop control, and DCOCLK are disabled
 - DCO's dc generator is disabled

- ACLK remains active
- Low-power mode 4 (LPM4)
 - CPU is disabled
 - ACLK is disabled
 - MCLK, FLL loop control, and DCOCLK are disabled
 - DCO's dc generator is disabled
 - Crystal oscillator is stopped
 - Complete data retention

We will discuss the specifics of these modes in the lectures. For the moment, here is the code to place the MCU into Low power mode 0, and wake the CPU only when the push button is pressed, LED1 is then toggled within the interrupt.

```

//*****
//  Description:  Software to demonstrate interrupt based digital input, with
//  Author:
//  Date:
//*****

#include "msp430x54x.h"

void main(void)
{
    // Stop WDT
    WDTCTL = WDTPW + WDTHOLD;

    // Setup P1.0 as an output port
    P1DIR = P1DIR | BIT0;
    P1SEL = P1SEL & ~BIT0;

    // Setup P2.6 as an input port
    P2DIR = P2DIR & ~BIT6;
    P2SEL = P2SEL & ~BIT6;

    // Enable the pull up resistor and set as a pullup
    P2REN = P2REN | BIT6;
    P2OUT = P2OUT | BIT6;

    // P2.6 Hi/lo edge
    P2IES |= BIT6;

    // P2.6 IFG cleared
    P2IFG &= ~BIT6;

    // P2.6 Interrupt Enable
    P2IE |= BIT6;

    __bis_SR_register(LPM0_bits + GIE);    // Enter LPM0, enable interrupts
    __no_operation();                      // For debugger
}

// Port 1 interrupt service routine
#pragma vector=PORT2_VECTOR
__interrupt void Port_2(void)
{
    // P1.0 = toggle LED 1

```

```
P1OUT ^= BIT0;

// P2.6 IFG cleared
P2IFG &= ~BIT6;
}
```

In this case the line

```
__bis_SR_register(LPM0_bits + GIE);    // Enter LPM0, enable interrupts
```

is used to place the CPU into a low power mode, and it enables interrupts.

Lab Question 8. Modify the above code to toggle LED 1 if switch s1 is pressed, and to toggle LED 2 if switch 2 is pressed. This should be performed using interrupts. The processor should be placed into low power mode 0. (Hint: This will require checking the flag within the interrupt handler to see which push button was pressed.)

Lab Question 9. Repeat the above, placing all of setup code for the LED and push button switches into an initialization function.

3.4 Next laboratory – Universal Asynchronous Receiver / Transmitter

In the next laboratory we will investigate how to perform serial communications between two MSP430F5438

Lab Question 10 Discuss the parameters that need to be agreed between two devices to allow serial communications using the RS232 protocol?

Although most users of Personal Computers would not be aware of this, clocks are at the heart of any synchronous digital system. CPUs require clocks to run the CPU since

asynchronous operation is not possible in computer processing (it presents many difficulties such as when the data to be processed from comes from different places at different speeds). In PCs, the selection of clock speeds is determined by various factors. Unless you are overclocking, you will never deal with them directly. Microcontrollers, on the other hand, are usually very flexible with clocks and require that the designer specify what clocks will be used and at what speeds. Usually this means that both hardware and software aspects of clocks be considered during the design stage.

Lab Question 11 From the data sheet for the MSP430F5438 and associated documentation, what are the names of the three clock signals available within the MSP430f5438?

4 Lab Reflection

In general, your lab write-up should indicate that you have acquired a better understanding of the topics treated by the laboratory assignment. You should write half a page of text that explains the following aspects in the box below. **Please create a cohesive piece of text and do not just provide unconnected sentences in response to the following aspects of your learning experience.**

Three new facts/concepts that you learnt while undertaking the lab.

The most useful thing that you learnt.

What understanding you already had of the material being explored.

How this laboratory experience relates to any other learning that you are also undertaking at the moment, or have undertaken in the past.

A statement of anything additional that you would like to explore in this area of work. Has the investigation given you any ideas about possible applications of this technology?

